

LENDING DECISION PLATFORM · GOOGLE CLOUD

Lending Decision Platform

Agentic credit decisioning on Google Cloud — with explainability, fairness and compliance built in.

Vertex AI

GKE

Argo Workflows

Gemini + Claude

Databricks · MLflow

Staff ML Engineer — project deep-dive

What this deck covers

1

The journey

Discovery to delivery

2

Architecture

Full GCP stack + cloud services

3

ML training

Argo pipeline, Vertex AI, MLflow

4

Agents + RAG

4 agents, Vertex Vector Search

5

Lifecycle & release

Experiment to prod, safely

6

Reliability

Agent, data/ML & system failures

The problem we set out to solve

Business context

Manual underwriting was slow, inconsistent between officers, and hard to audit. We needed automated credit decisioning that is fast, consistent, explainable, and compliant with responsible-lending obligations.

The shape of the solution

A calibrated ML lending-decision model at the core, wrapped in four specialised AI agents (application intake, KYC/AML, financial analysis, security audit), grounded in current lending policy via RAG, with a human in the loop on high-value / low-confidence cases — all on Google Cloud.

minutes

decision turnaround vs. hours of manual review

Reason codes

on every decision — adverse-action ready & auditable

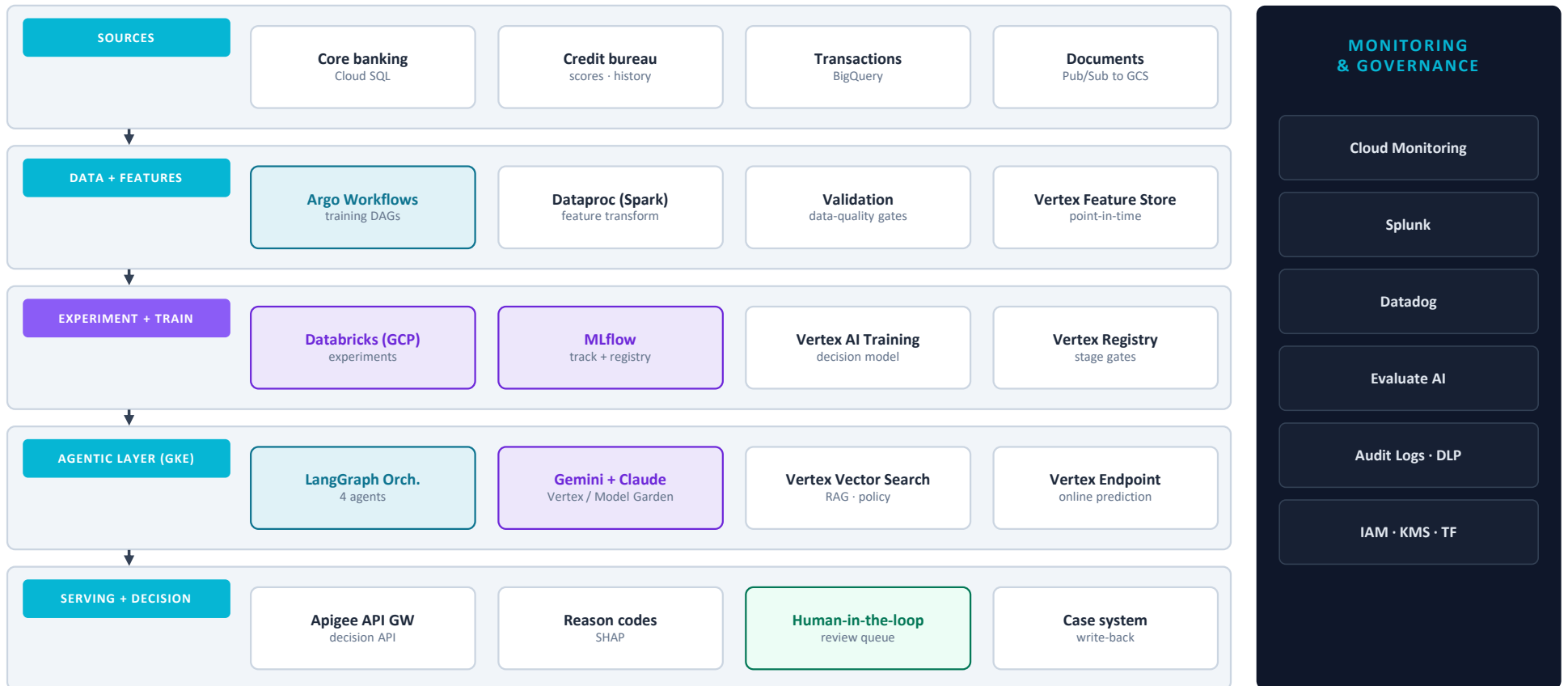
HITL gate

humans review high-value & low-confidence cases

How it unfolded: discovery to delivery



Solution architecture — end to end with cloud services



Google Cloud services used across the platform

● Compute & Orchestration

Google Kubernetes Engine (GKE)
Argo Workflows on GKE
Cloud Run

● Data & Analytics

BigQuery
Cloud Storage (GCS)
Pub/Sub
Dataproc (Spark) · Dataflow
AlloyDB / Cloud SQL

● ML Platform — Vertex AI

Vertex AI Training
Vertex AI Endpoints
Vertex Feature Store
Vertex Model Registry
Vertex AI Pipelines

● LLM & RAG

Gemini (Vertex AI)
Claude (Model Garden)
Vertex AI Vector Search

● CI/CD & IaC

Cloud Build
Artifact Registry
Terraform

● Security & Governance

IAM · Cloud KMS
VPC-SC · DLP
Secret Manager
Cloud Audit Logs

● Monitoring & Alerting

Cloud Monitoring / Logging
Splunk · Datadog
Evaluate AI (LLM eval)

How model training works — with cloud services



Orchestrated by Argo Workflows on GKE — every run idempotent, versioned, reproducible (MLflow)

Retraining is triggered, not scheduled by habit. Drift monitors or accumulated label corrections fire a run; the gate blocks promotion unless metric, fairness and calibration checks pass.

Point-in-time features

One Vertex Feature Store definition serves training and the live endpoint — no skew, no leakage.

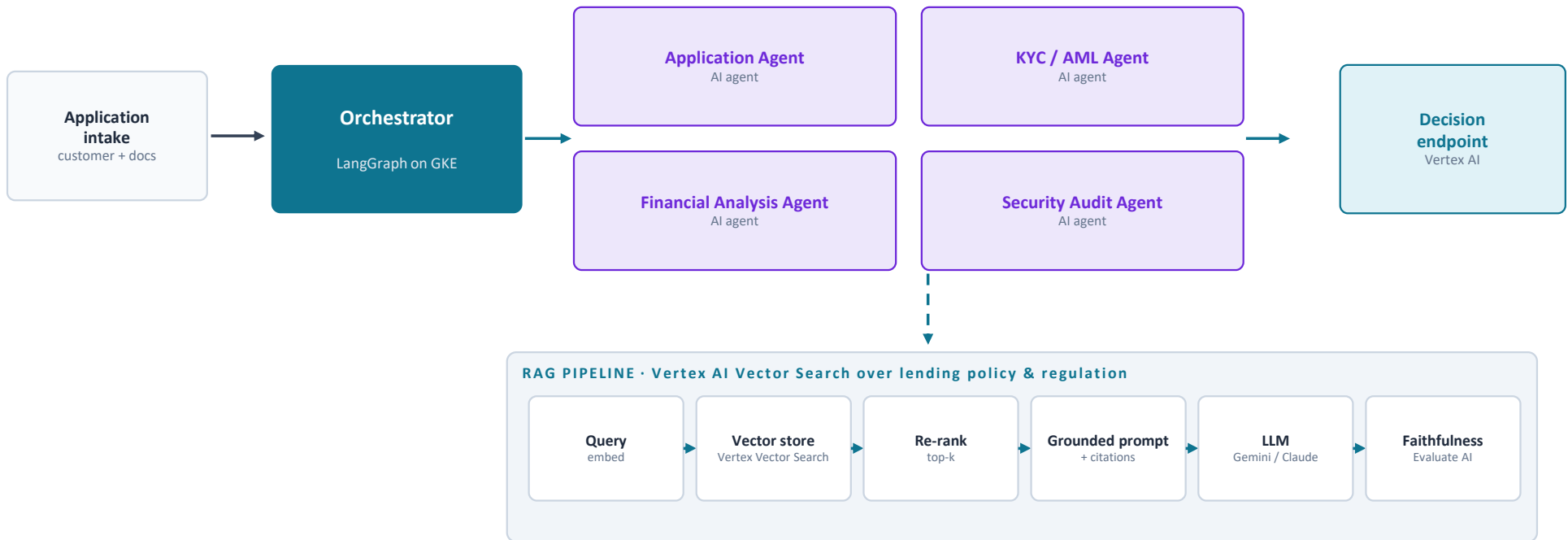
Fairness as a gate

Sliced error + adverse-impact checks run in CI, not as a post-hoc audit.

Calibrated probabilities

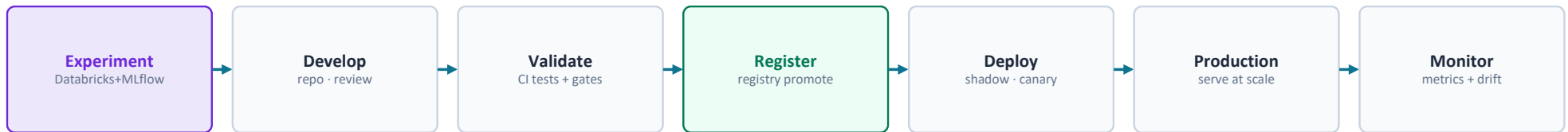
Decisions ride on probabilities, so isotonic/Platt calibration is mandatory.

AI agents + RAG pipeline — with cloud services



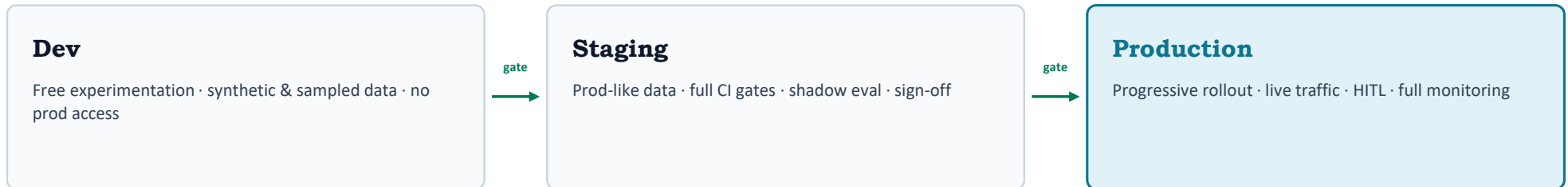
Agents call the ML scoring endpoint + data tools, then return a grounded, cited recommendation -> reason codes -> human-in-the-loop for the edge cases.

Experiments -> development -> deployment -> production



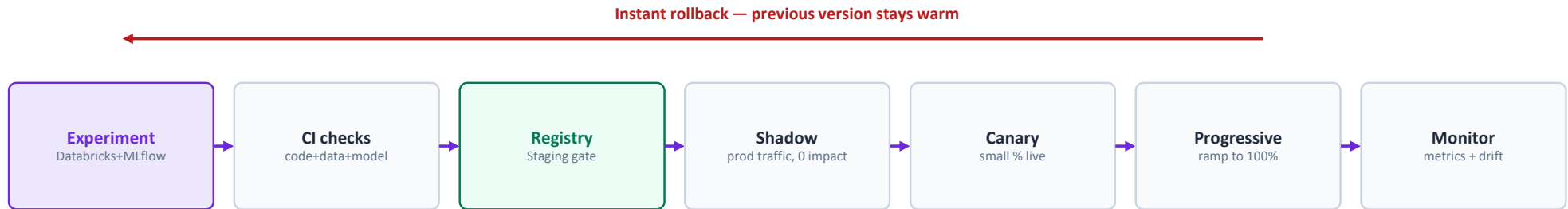
Monitoring closes the loop — drift / decay opens the next experiment

PROMOTION ACROSS ENVIRONMENTS

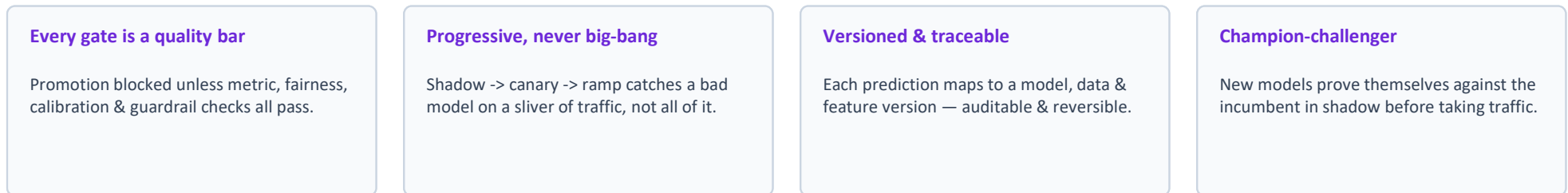


Each promotion is gated; artifacts (code, data snapshot, feature & model versions) are immutable and traceable end to end via MLflow + the model registry.

Shipping fast without compromising production



Drift / decay -> triggers a new experiment (closed loop)



Monitoring & measuring ML metrics and accuracy



Operational health

Cloud Monitoring + Datadog

Latency p50/p99, throughput, error rate, saturation, cost per 1k predictions



Data & feature drift

PSI / KS monitors

Input & prediction-distribution shift, feature freshness & null spikes



Model performance

Sliced metrics

AUC, calibration, override rate and fairness by segment — tracked as labels arrive



LLM / agent quality

Evaluate AI

Faithfulness, citation validity, refusal / leak checks on every agent output

Splunk aggregates logs & fires alerts across all four layers. Alert thresholds are tied to business impact, not arbitrary statistics — minor drift logs & watches, sustained decay triggers retraining, a sharp break rolls back. Ground-truth lending performance is tracked as repayment labels arrive, so we measure real accuracy, not just proxies.

ML development strategy — MLflow, TensorFlow & PyTorch

MLflow — the connective tissue. One tracking + registry layer across frameworks and clouds: every run's params, data & feature versions, metrics and artifacts are logged; the registry governs Staging-to-Production promotion with the release gates. Runs on Databricks; models serve on Vertex AI.

PyTorch · research & iteration

- Fast, flexible experimentation — default for research & rapid iteration
- Where we fine-tune transformer / LLM components
- Dynamic graphs make debugging & custom logic easy
- Served via TorchServe or exported to ONNX / Vertex endpoints

TensorFlow · production serving

- Mature serving & lifecycle ecosystem (TF Serving, TFX)
- Preferred for a hardened, high-throughput serving path
- Strong fit for managed Vertex AI endpoints
- Graph optimisation & quantisation for low-latency inference

The strategy in one line: *prototype freely in PyTorch, standardise artifacts & metrics through MLflow, then serve on Vertex AI — and for tabular credit decisioning a calibrated gradient-boosted model still wins, which is a feature, not a compromise.*

What I owned vs. what the team delivered

What I personally owned · Staff scope

- Set the technical direction & full solution architecture
- ML development & deployment: model, calibration, evaluation, Vertex serving
- AI/agent development & deployment: orchestration, tool contracts, guardrails
- Designed the Argo training pipeline + Vertex Feature Store (train=serve)
- Designed the RAG grounding + Evaluate AI faithfulness gating
- Established MLOps standards: MLflow registry, CI/CD, progressive release gates
- Mentored 5 engineers; ran the design reviews that set the bar

What the team delivered

- Built source-system ingestion connectors & data contracts
- Implemented individual agent logic against the framework I defined
- Implemented model training code from the pipeline templates
- Ran GKE platform / SRE & on-call operations
- Credit analysts validated decision quality & reason codes
- QA & compliance executed UAT and sign-off

Maps to the role:

Architecture

ML dev & deploy

AI dev & deploy

System design

Standards

Mentoring

Client & stakeholder intersection



Lending & Credit Risk

Co-owned the decision policy; I translated their rules into model logic, thresholds & reason codes.



Compliance & Legal

Responsible-lending obligations to explainability, fairness testing & audit requirements I designed to.



Model Risk Review

Presented model methodology & validation for formal sign-off before each promotion.



Product & Underwriters

Designed the human-in-the-loop review flow with the people who'd use it.



Data Governance

Agreed data lineage, access controls & retention for regulated financial data.



Cloud / Google

Architecture reviews on Vertex AI & GKE; negotiated managed-service trade-offs.

Technology decisions & trade-offs

Decision	What I chose	Trade-off I accepted
Decision model class	Calibrated gradient-boosted trees	Less cutting-edge than deep nets — but explainable, fast, regulator-friendly on tabular data
Serving	Vertex AI managed endpoints	Some GCP lock-in, in exchange for built-in monitoring & faster, safer delivery
Training orchestration	Argo Workflows on GKE	More ops than Vertex Pipelines, bought portability & k8s-native flexibility
Policy knowledge	RAG over fine-tuning	More moving parts, but fresh, citable policy — essential for compliance
LLMs	Gemini + Claude, task-routed	Multi-vendor complexity, in exchange for best-fit quality/cost & no lock-in

Challenges & how I worked through them

Decisions had to be explainable & fair

Interpretable model + SHAP reason codes + adverse-action notices; fairness tested across protected attributes with a full audit trail.

LLM hallucination in financial analysis

Grounded every agent via RAG, verified numeric claims against source data, and gated outputs through the Evaluate AI faithfulness harness.

Multi-agent latency vs thoroughness

Ran agents in parallel, cached retrievals & embeddings, held a strict SLA budget with graceful degradation.

Disparate, messy source data

Data-quality gates at every boundary + a feature store as the single source of truth.

Addressing AI & agent failure modes

● Hallucination / unfaithful output

How we contain it: ground every claim via RAG with citations; verify numeric claims against source data; an Evaluate AI faithfulness gate + confidence threshold blocks risky answers; fall back to a human on low confidence.

● Bad arguments passed to tools

How we contain it: strict typed tool schemas with JSON-schema validation; arguments validated & sanitised before any call; auto-repair retry on validation failure; the agent asks rather than guesses when inputs are ambiguous.

● Wrong tool selected

How we contain it: task-scoped tool allow-lists; clear tool specs + few-shot examples; a planner-critic review step; deterministic routing where simple rules suffice; tool-choice accuracy tracked in the eval suite.

● Context overload

How we contain it: retrieve, don't stuff — only top-k relevant chunks; rolling summarisation / compaction of history; structured memory; hard per-agent token budgets with relevance filtering.

Handling data / ML & system / tool failures

DATA / ML

Data-quality drift

Schema + distribution checks (PSI/KS) at every boundary; anomaly detection; a circuit-breaker halts promotion & serving on bad data.

DATA / ML

Concept drift

Label-based performance monitors + drift detectors trigger retraining; champion-challenger validates before any cutover.

DATA / ML

Missing feature values

Vertex Feature Store with versioned defaults & imputation; null-rate monitoring; safe fallbacks keep serving online in a flagged degraded mode.

SYSTEM / TOOL

API timeouts

Timeouts + exponential-backoff retries, circuit breakers, bulkheads, caching and async calls with a graceful fallback path.

SYSTEM / TOOL

Schema mismatches

Schema registry + contract tests in CI; versioned, backward-compatible APIs; validation at every boundary before execution.

SYSTEM / TOOL

Orchestration bottlenecks

Parallelise independent agents; concurrency limits + backpressure queues; rate limiting; idempotent steps; distributed tracing to find hot spots.

WHAT THIS JOURNEY SHOWS

The through-line

● **Production ML at scale**

Feature store, real-time Vertex serving, drift-aware retraining — ML that holds up in production.

● **Principled trade-offs**

Explainable GBMs over flashy deep nets; managed vs self-managed on merit; metrics tied to business cost.

● **Resilient by design**

Concrete containment for agent, data/ML and system failures — not hope.

● **Standards & mentoring**

I set the patterns, ran the design reviews, and grew the engineers who built within them.

The research-meets-production seam, at scale, for a mission that matters — exactly what I want to do next.